

Assignment 2 – Case Study

Stage 4 Tutorial Activities

Object-Oriented Design Decisions

Week 7 | 60 minutes

Activity 1 - Encapsulation Review

Class	Protected state / invariant	Public operations
Patient	id, name, contact_info (must not be left empty)	get_history()
Practitioner	Id, name, specialty	get_schedule()
Appointment	status	cancel()

Activity 2 - Composition or Inheritance?

Appointment and Patient -> **Composition/association** Inheritance Reason: An appointment is an event that involves a patient, it is not a sub-type of a patient.

Appointment and Practitioner -> **Composition/association** Inheritance Reason: An appointment is assigned to a practitioner, it doesn't inherit from one.

Doctor and Practitioner (hypothetical) -> **Composition/association** **Inheritance** Reason: A doctor is a specific type of Practitioner (this is an "is-a" relationship).

Clinic and Appointment -> **Composition/association** Inheritance Reason: A clinic contains or hosts appointments, it doesn't inherit from them.

Activity 3 - Responsibility Allocation

Who decides whether SCHEDULED can become CANCELLED?

The Appointment class itself, inside its cancel() method.

Who validates a patient name?

The Patient class, inside its constructor (__init__).

Should Appointment execute SQL? Why?

No. The domain layer should only hold business logic. Database operations belong in a separate data access layer to keep the system maintainable.

Should the UI decide whether a status transition is legal?

No. The core domain object (Appointment) must enforce its own rules so bad data can never be saved, regardless of the UI.

Activity 4 - AI Code Critique

1. Public status mutation: The status shouldn't be publicly editable. Correction: Make status private and use a dedicated cancel() method.
2. SQL inside cancel(): Domain classes shouldn't talk to databases. Correction: Remove SQL entirely from the domain model.
3. NotificationManager dependency: This is a service/infrastructure concern. Correction: Remove the NotificationManager from the core domain class.
4. Inheritance from PatientRecord: An Appointment is not a type of PatientRecord. Correction: Use association (linking) instead of inheritance.
5. Missing invariant protection: Public mutation means anyone can change it to anything. Correction: Add if-statements in a method to block illegal status changes.

Exit question

Code can use objects and classes but can still be essentially one giant script if it fails to use proper encapsulation, separates data from behaviour, or ignores single responsibility. Good design means objects protect their own state and only do the specific job they were created for.